



Master's in Informatics and Multimedia Engineering

Computer Vision and Mixed Reality

2nd Project

Marker Based Augmented Reality

1. Goal

- a. Develop a computer vision application that implements marker-based augmented reality that allows the inclusion of virtual elements aligned with real fiducial markers.
- b. Familiarization with the OpenCV (**Open**-Source **C**omputer **V**ision) library for real-time application programming and the ArUco marker-based library.

2. Development

a. Camera Calibration

Camera calibration is the process of computing the intrinsic parameters of the camera – the Perspective Transformation – and the distortion coefficients (optional) for correcting the radial and tangential distortions caused by the camera lens.

OpenCV Camera Calibration Tutorial [\[1\]](#) demonstrates how to implement this task using built-in OpenCV functions. However, there are alternative methods using other libraries as described, for example, in [\[2, 3\]](#).

Create an application with the method proposed in [\[1\]](#) (or choose another one) to calibrate your computer camera offline and save the intrinsic parameters for future use.

b. Detection and camera pose estimation with ArUco library

ArUco library for OpenCV [\[3, 4\]](#) is a marker-based “minimal library for Augmented Reality applications based on OpenCV”.

Create an application to detect ArUco markers and estimate the camera pose for each marker – View Transformation – to enable the registration of virtual objects aligned with the markers. The ArUco markers can be generated using the proper method, but [\[5\]](#) provides an ArUco marker generator. A Python example with the ArUco library can be found in [\[6\]](#).

c. Registration of virtual objects in OpenCV

Add virtual objects registered with the world coordinate system associated with the detected markers (different objects for each marker ID).

d. Integration with Unity3D (Optional)

Integrate the augmented reality application with the Unity3D game engine [\[7\]](#) to render more sophisticated virtual objects.

The integration should be based on a client-server architecture in which the Python/OpenCV application serves as the server, and Unity3D acts as the client. Communication between the two applications should be via UDP sockets.

The Python/OpenCV application should:

- Capture the camera image in real-time
- Detect the ArUco markers and estimate the camera pose
- Send the camera image (JPEG compressed) to Unity3D via UDP (port 5000)
- Send the marker pose data (JSON format) to Unity3D via UDP (port 5001)

The Unity3D application should:

- Receive the camera image and display it as a background in real-time
- Receive the marker pose data and position a 3D virtual object aligned with the detected marker
- The virtual camera intrinsic parameters should match the real camera parameters obtained from the calibration step

Bibliography

- [1] - https://docs.opencv.org/master/dc/dbb/tutorial_py_calibration.html
- [2] - <https://medium.com/vacatronics/3-ways-to-calibrate-your-camera-using-opencv-and-python-395528a51615>
- [3] - <https://www.uco.es/investiga/grupos/ava/portfolio/aruco/>
- [4] - https://docs.opencv.org/4.9.0/d9/d6d/tutorial_table_of_content_aruco.html
- [5] - <https://chev.me/arucogen/>
- [6] - https://mecaruco2.readthedocs.io/en/latest/notebooks_rst/Aruco/aruco_basics.html
- [7] - <https://docs.unity3d.com/Manual/index.html>

Main *OpenCV* function for camera calibration and pose estimation:

Goal	OpenCV function(s)
Finds the positions of internal corners of the chessboard.	<i>findChessboardCorners()</i>
Renders the detected chessboard corners.	<i>drawChessboardCorners()</i>
Finds subpixel-accurate positions of the chessboard corners.	<i>cornerSubPix()</i>
Finds the camera intrinsic and extrinsic parameters from several views of a calibration pattern.	<i>calibrateCamera()</i>
Returns the new camera intrinsic matrix based on the free scaling parameter.	<i>getOptimalNewCameraMatrix()</i>
Transforms an image to compensate for lens distortion.	<i>undistort()</i>
Finds an object pose from 3D-2D point correspondence using the RANSAC scheme.	<i>solvePnPRansac()</i>
Project 3D points to an image plane.	<i>projectPoints()</i>
Convert a rotation matrix to a rotation vector or vice versa.	<i>Rodrigues()</i>
Draw the coordinate system axis from the pose estimation.	<i>drawFrameAxes()</i>
Draw a canonical marker image.	<i>drawMarker()</i>

Main *OpenCV.ArUco* functions for marker generation, detection, and pose estimation:

Goal	OpenCV function(s)
Returns one of the predefined ArUCo dictionaries.	<i>getPredefinedDictionary()</i>
Load the ArUCo parameters for the detectMarker process.	<i>DetectorParameters()</i>
Basic ArUcoDetector constructor for marker detection.	<i>ArucoDetector()</i>
Basic marker detection.	<i>detectMarkers()</i>
Draw detected markers in image.	<i>drawDetectedMarkers()</i>
Pose estimation for single markers.	<i>estimatePoseSingleMarkers()</i>